

Combined Topology Statistics Specification

Normative TS4 API for Atomic-State/Framework-Class Alignment and Cross-Layer Transition
Classification in mdstats

mdstats

2026-07-14 (implemented TS4 revision)

Contents

1 Purpose and status	2
2 Motivation	3
3 Scientific responsibility boundaries	3
3.1 Owned by <code>combined.py</code>	3
3.2 Not owned by <code>combined.py</code>	4
4 Module relationships	4
5 Public schema constants	4
6 Public API summary	4
7 Input contract	5
7.1 Atomic catalog	5
7.2 Framework catalog	5
7.3 Frame axis	6
7.4 Options	6
8 Exact catalog alignment	6
9 Contingency theory	7
10 Contingency result model	7
10.1 <code>AtomicStateProjectionStatistics</code>	7
10.2 <code>FrameworkClassCompositionStatistics</code>	7
10.3 <code>CrossLayerContingencyStatistics</code>	8
11 Trajectory boundary classification	8
11.1 Eligibility	8
11.2 <code>CrossLayerBoundaryEvent</code>	9
11.3 <code>CrossLayerBoundaryStatistics</code>	9
12 Compact cross-layer summary	9
12.1 Regime	9
12.2 Summary object	9
13 Combined result	10

14 Algorithm	10
14.1 High-level procedure	10
14.2 Pseudocode	10
15 Complexity	11
16 Serialization	11
17 Provenance	12
18 Errors	12
18.1 TopologyStatisticsInputError	12
18.2 TopologyStatisticsConsistencyError	12
18.3 TopologyStatisticsSerializationError	12
19 Edge cases and warnings	12
19.1 Many atomic states mapping to one framework class	12
19.2 One atomic state mapping to several framework classes	12
19.3 Framework-only boundaries	12
19.4 Atomic-only does not mean spectator-only	13
19.5 Atomic transition does not imply site hopping	13
19.6 Ensemble contingency is not a transition model	13
19.7 Per-frame identity modes	13
19.8 Compression ratio	13
20 Validation requirements	13
21 Na-LTA acceptance case	13
22 Deferred features	14
23 Context-restoration checklist	14
24 Final contract	14

1 Purpose and status

This document specifies the implemented module

`mdstats/analysis/topology_statistics/combined.py`

and its public re-export surface through

```
mdstats.analysis.topology_statistics
mdstats.analysis
mdstats
```

for topology-statistics Stage TS4. The implementation is introduced in `mdstats 0.17.0a4`.

TS4 consumes one authoritative `AtomicConnectivityResult` and the `TopologyCatalog` projected from the same selected frames. It invokes the TS1 atomic and TS2 framework statistics branches, then derives only cross-layer alignment statistics.

The governing boundary is

atomic and framework catalogs define identity; TS4 compares their assignments.

TS4 provides:

- exact validation that the two catalogs refer to the same selected frames;

- exact validation that framework connectivity-state assignments match the atomic catalog;
- one atomic-state/framework-class contingency matrix;
- conditional framework-class composition of every atomic state;
- conditional atomic-state composition of every framework class;
- atomic-to-framework compression metrics;
- exact adjacent-boundary classification for reconciled trajectories;
- a compact cross-layer regime and interpretation string;
- one combined immutable result containing the complete TS1 and TS2 branches;
- schema-versioned serialization and SHA-256 digest validation.

TS4 does not infer chemical mechanisms, site hops, ring changes, reaction rates, or approximate state similarity.

2 Motivation

Atomic connectivity and projected framework topology answer different scientific questions. Mobile spectators may alter the atomic contact graph without changing the structural framework. Conversely, a genuine framework event should be visible at both layers.

For frame f , let

$$s_f^A$$

be the atomic-connectivity state ID and

$$s_f^F$$

be the projected framework topology-class ID.

A combined analysis should answer:

- How many atomic states map to each framework class?
- Does one atomic state always project to one framework class?
- Which atomic transitions preserve the framework?
- Which atomic transitions coincide with a framework transition?
- Are the two catalogs exactly aligned, or merely similar in length?
- Can the central scientific statement be produced automatically?

For the validated 300 K Na-LTA trajectory, TS4 reports:

```
72 atomic-connectivity states
1 framework topology class
71 framework-preserving atomic transition boundaries
0 framework-changing atomic transition boundaries
```

with the interpretation:

```
atomic connectivity varies while framework topology remains uniform
```

3 Scientific responsibility boundaries

3.1 Owned by `combined.py`

The module owns:

- exact atomic/framework source alignment validation;
- TS1 and TS2 branch invocation through one common frame axis;
- frame contingency between atomic states and framework classes;
- conditional state/class composition summaries;

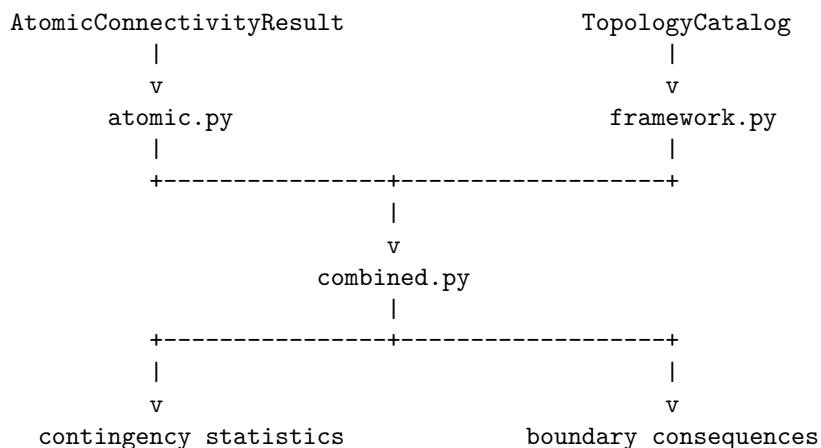
- atomic-to-framework compression metrics;
- trajectory boundary consequence classification;
- compact cross-layer regime classification;
- combined provenance, serialization, and digest validation.

3.2 Not owned by combined.py

The module must not:

- rebuild neighbors or atomic connectivity;
- choose connectivity cutoffs;
- project atomic paths into framework edges;
- reconcile new framework topology classes;
- smooth or merge states;
- classify a Na–O contact change as a site hop;
- infer which chemical mechanism caused a transition;
- enumerate primitive rings;
- compute transition probabilities, rates, or Markov models;
- treat ensemble storage order as time.

4 Module relationships



The combined module invokes existing branch functions. It does not duplicate their graph-specific calculations.

5 Public schema constants

```
CANONICAL_COMBINED_TOPOLOGY_STATISTICS_SCHEMA = (
    "mdstats.topology-statistics.combined.v1"
)
```

The package-wide statistics digest algorithm is

sha256

6 Public API summary

```
class CrossLayerBoundaryKind(str, Enum): ...
class CrossLayerCatalogRegime(str, Enum): ...

@dataclass(frozen=True, slots=True)
class CombinedStatisticsOptions: ...
```

```

@dataclass(frozen=True, slots=True)
class AtomicStateProjectionStatistics: ...

@dataclass(frozen=True, slots=True)
class FrameworkClassCompositionStatistics: ...

@dataclass(frozen=True, slots=True)
class CrossLayerContingencyStatistics: ...

@dataclass(frozen=True, slots=True)
class CrossLayerBoundaryEvent: ...

@dataclass(frozen=True, slots=True)
class CrossLayerBoundaryStatistics: ...

@dataclass(frozen=True, slots=True)
class CrossLayerSummary: ...

@dataclass(frozen=True, slots=True)
class TopologyStatistics: ...

def compute_topology_statistics(
    atomic_catalog: AtomicConnectivityResult,
    framework_catalog: TopologyCatalog,
    *,
    steps: ArrayLike | None = None,
    times: ArrayLike | None = None,
    time_unit: str | None = None,
    options: CombinedStatisticsOptions | None = None,
) -> TopologyStatistics:
    ...

```

7 Input contract

7.1 Atomic catalog

`atomic_catalog` must be one completed `AtomicConnectivityResult`.

Required properties include:

- nonempty selected frame arrays;
- stable canonical atom identity across selected frames;
- valid `frame_semantics` in catalog metadata;
- dense valid `frame_state_ids`;
- immutable canonical connectivity states;
- source state digests.

7.2 Framework catalog

`framework_catalog` must be one completed `TopologyCatalog` built from the same selected atomic catalog or an exactly equivalent source with identical state IDs and state digests.

Required properties include:

- identical selected frame indices;

- identical frame IDs and ordering;
- identical frame semantics;
- identical frame-to-connectivity-state assignments;
- a valid framework mapping and topology schema;
- topology representatives whose source connectivity digests belong to the aligned atomic states.

Equal frame count alone is insufficient.

7.3 Frame axis

Optional `steps`, `times`, and `time_unit` are passed identically to TS1 and TS2.

For trajectories:

- `steps`, when supplied, must be strictly increasing integer values;
- `times`, when supplied, must be finite and strictly increasing;
- `time_unit` is required with `times`;
- `time_unit` is forbidden without `times`.

For ensembles, all three are forbidden because stored sample order has no temporal meaning.

7.4 Options

```
@dataclass(frozen=True, slots=True)
class CombinedStatisticsOptions:
    atomic_options: AtomicStatisticsOptions = AtomicStatisticsOptions()
    framework_options: FrameworkStatisticsOptions = FrameworkStatisticsOptions()
    include_boundary_statistics: bool = True
```

The nested options are passed unchanged to TS1 and TS2.

8 Exact catalog alignment

TS4 validates the following equalities:

$$\mathbf{f}^A = \mathbf{f}^F,$$

where $\mathbf{f}$ is the selected collection-frame-index array,

$$\mathbf{id}^A = \mathbf{id}^F,$$

where $\mathbf{id}$ is the frame-ID array, and

$$\mathbf{s}^A = \mathbf{s}^{F, \text{source}},$$

where $\mathbf{s}^{F, \text{source}}$ is the connectivity-state assignment stored inside the framework catalog.

For reconciled catalog mode, the framework state-to-topology map must cover every atomic state. Each topology representative must carry the digest of one atomic state mapped to that topology.

For per-frame topology mode, each public per-frame topology must carry the digest of the atomic state assigned to the same frame.

The alignment mode is reported as one of:

```
exact_catalog
exact_per_frame
```

9 Contingency theory

For atomic state a and framework class k , the frame contingency count is

$$C_{ak} = \sum_{f=1}^F \mathbf{1}[s_f^A = a] \mathbf{1}[s_f^F = k].$$

The joint empirical probability is

$$P_{ak} = \frac{C_{ak}}{F}.$$

The conditional probability that atomic state a projects to framework class k is

$$P(k | a) = \frac{C_{ak}}{\sum_j C_{aj}}.$$

The conditional probability that a frame in framework class k has atomic state a is

$$P(a | k) = \frac{C_{ak}}{\sum_i C_{ik}}.$$

The atomic-to-framework compression ratio is

$$R_{A/F} = \frac{N_A}{N_F},$$

where N_A is the number of atomic states and N_F is the number of framework classes.

This ratio measures catalog compression, not information-theoretic loss or a kinetic coarse-graining quality.

10 Contingency result model

10.1 AtomicStateProjectionStatistics

```
@dataclass(frozen=True, slots=True)
class AtomicStateProjectionStatistics:
    atomic_state_id: int
    framework_class_ids: NDArray[np.int64]
    frame_counts: NDArray[np.int64]
    conditional_probabilities: NDArray[np.float64]
    dominant_framework_class_ids: tuple[int, ...]
```

`framework_class_ids` contains only classes with nonzero occupancy for the atomic state and is strictly increasing.

`is_deterministic` is true when the atomic state occurs in exactly one framework class.

10.2 FrameworkClassCompositionStatistics

```
@dataclass(frozen=True, slots=True)
class FrameworkClassCompositionStatistics:
    framework_class_id: int
    atomic_state_ids: NDArray[np.int64]
    frame_counts: NDArray[np.int64]
    conditional_probabilities: NDArray[np.float64]
    dominant_atomic_state_ids: tuple[int, ...]
```

This is the column-wise counterpart of the atomic-state projection result.

10.3 CrossLayerContingencyStatistics

```
@dataclass(frozen=True, slots=True)
class CrossLayerContingencyStatistics:
    frame_count_matrix: NDArray[np.int64]
    probability_matrix: NDArray[np.float64]
    atomic_state_projections: tuple[AtomicStateProjectionStatistics, ...]
    framework_class_compositions: tuple[FrameworkClassCompositionStatistics, ...]
```

Convenience properties include:

```
n_frames
n_atomic_states
n_framework_classes
atomic_to_framework_compression_ratio
atomic_states_per_framework_class
framework_classes_per_atomic_state
```

All arrays are defensively copied and read-only.

11 Trajectory boundary classification

For adjacent result positions $f - 1$ and f , define

$$\Delta_A(f) = \mathbf{1}[s_f^A \neq s_{f-1}^A],$$

and

$$\Delta_F(f) = \mathbf{1}[s_f^F \neq s_{f-1}^F].$$

Every boundary belongs to exactly one class:

Atomic change	Framework change	CrossLayerBoundaryKind
0	0	STABLE
1	0	ATOMIC_ONLY
0	1	FRAMEWORK_ONLY
1	1	COUPLED

FRAMEWORK_ONLY is retained as an explicit diagnostic. Under exact deterministic catalog projection it should not occur, because one unchanged atomic state must project to one unchanged reconciled topology. A nonzero value indicates an unusual per-frame identity policy, inconsistent source data, or a future extension with additional frame-dependent projection input.

11.1 Eligibility

Boundary statistics are generated only when:

- frame semantics are TRAJECTORY;
- the atomic catalog has reconciled state identity rather than PER_FRAME identity;
- the framework catalog uses reconciled catalog mode;
- include_boundary_statistics=True.

For ensembles, per-frame atomic catalogs, or per-frame framework catalogs, TS4 still returns static contingency statistics but does not invent transition semantics.

11.2 CrossLayerBoundaryEvent

Only non-stable boundaries are materialized as event objects. Each event records:

- adjacent result positions;
- collection frame indices;
- frame IDs;
- source and target atomic state IDs;
- source and target framework class IDs;
- boundary kind;
- optional step and physical-time pairs.

11.3 CrossLayerBoundaryStatistics

```
@dataclass(frozen=True, slots=True)
class CrossLayerBoundaryStatistics:
    axis: FrameAxis
    boundary_kind_codes: NDArray[np.int64]
    events: tuple[CrossLayerBoundaryEvent, ...]
```

The dense code array contains one entry per adjacent boundary. The canonical code order is:

```
0 stable
1 atomic_only
2 framework_only
3 coupled
```

Convenience counts include:

```
n_stable_boundaries
n_atomic_only_boundaries
n_framework_only_boundaries
n_coupled_boundaries
n_atomic_changed_boundaries
n_framework_changed_boundaries
```

12 Compact cross-layer summary

12.1 Regime

```
class CrossLayerCatalogRegime(str, Enum):
    UNIFORM = "uniform_atomic_and_framework"
    ATOMIC_VARIABLE_FRAMEWORK_UNIFORM = "atomic_variable_framework_uniform"
    FRAMEWORK_VARIABLE = "framework_variable"
```

The regime is descriptive and depends only on the number of atomic states and framework classes.

12.2 Summary object

```
@dataclass(frozen=True, slots=True)
class CrossLayerSummary:
    regime: CrossLayerCatalogRegime
    n_atomic_states: int
    n_framework_classes: int
    atomic_to_framework_compression_ratio: float
    n_atomic_changed_boundaries: int | None
    n_framework_changed_boundaries: int | None
    n_framework_preserving_atomic_boundaries: int | None
    n_framework_changing_atomic_boundaries: int | None
```

For non-temporal results, all boundary fields are `None`.

The `interpretation` property returns one neutral descriptive sentence. It does not claim a chemical mechanism.

13 Combined result

```
@dataclass(frozen=True, slots=True)
class TopologyStatistics:
    atomic: AtomicConnectivityStatistics
    framework: FrameworkTopologyStatistics
    contingency: CrossLayerContingencyStatistics
    boundary_statistics: CrossLayerBoundaryStatistics | None
    summary: CrossLayerSummary
    options: CombinedStatisticsOptions
    alignment_mode: str
    metadata: Mapping[str, Any]
    canonical_schema_version: str
    digest_algorithm: str
    digest: str
```

The atomic and framework branches are complete TS1/TS2 result objects. TS4 does not create a second competing representation of their distributions or timelines.

14 Algorithm

14.1 High-level procedure

```
validate input types
validate exact frame, ID, semantics, and state-source alignment
compute TS1 atomic statistics on the requested axis
compute TS2 framework statistics on the same axis
verify identical resulting FrameAxis objects
accumulate atomic-state/framework-class contingency counts
build row-wise and column-wise conditional summaries
if reconciled trajectory semantics are available:
    classify every adjacent frame boundary
else:
    record why boundary statistics are unavailable
build compact cross-layer summary
construct immutable schema-versioned result
```

14.2 Pseudocode

```
function compute_topology_statistics(A, F, axis_options, options):
    validate_exact_alignment(A, F)

    atomic_stats = compute_atomic_connectivity_statistics(A, axis_options)
    framework_stats = compute_framework_topology_statistics(F, axis_options)

    C = zeros(A.n_states, F.n_topologies)
    for frame_position in analyzed_frames:
        a = A.frame_state_ids[frame_position]
        k = F.frame_topology_ids[frame_position]
        C[a, k] += 1

    contingency = summarize_rows_and_columns(C)
```

```

if exact_reconciled_trajectory(A, F):
    kinds = classify_adjacent_boundaries(A.frame_state_ids,
                                         F.frame_topology_ids)
    boundaries = build_boundary_result(kinds)
else:
    boundaries = None

return TopologyStatistics(...)

```

15 Complexity

Let:

- F be the number of selected frames;
- N_A be the number of atomic states;
- N_F be the number of framework classes.

The TS4-specific contingency and boundary work scales as

$$O(F + N_A N_F)$$

in time and

$$O(N_A N_F + F)$$

in memory when dense contingency and boundary code arrays are retained.

The graph-descriptor costs belong to TS1 and TS2 and exploit catalog compression. TS4 does not reprocess atomic or framework edge sets.

For unusually large $N_A N_F$, a future sparse contingency representation may be useful. The initial dense representation is deliberate because common workflows have many fewer states/classes than frames and benefit from simple deterministic serialization.

16 Serialization

`TopologyStatistics.to_dict()` returns a JSON-compatible payload containing:

- combined schema and object type;
- digest algorithm and digest;
- complete serialized atomic statistics;
- complete serialized framework statistics;
- contingency result;
- optional boundary result;
- compact summary;
- nested options;
- alignment mode and metadata.

`TopologyStatistics.from_dict()` validates nested TS1 and TS2 digests before validating the combined digest.

A stored digest mismatch raises `TopologyStatisticsConsistencyError`. Malformed or unsupported payloads raise `TopologyStatisticsSerializationError`.

17 Provenance

Metadata records at least:

- frame semantics;
- boundary-statistics status and omission reason;
- atomic catalog consistency category;
- framework catalog consistency category;
- framework catalog mode;
- source atomic state digests;
- source framework catalog digest;
- source framework mapping digest.

The authoritative source objects remain the atomic and framework catalogs.

18 Errors

The module uses the shared topology-statistics error hierarchy.

18.1 TopologyStatisticsInputError

Raised for incompatible source catalogs, including:

- mismatched frame semantics;
- mismatched selected frame indices;
- mismatched frame IDs or ordering;
- mismatched connectivity-state assignments;
- incomplete state-to-topology mappings;
- topology representatives not derived from aligned atomic states.

18.2 TopologyStatisticsConsistencyError

Raised when a constructed or deserialized result violates its internal invariants.

18.3 TopologyStatisticsSerializationError

Raised for malformed or unsupported serialized payloads.

19 Edge cases and warnings

19.1 Many atomic states mapping to one framework class

This is a normal and important result for mobile spectators. It should not be reported as a framework instability.

19.2 One atomic state mapping to several framework classes

In reconciled deterministic catalog mode this should not occur. It may occur in per-frame framework mode because public topology IDs are intentionally not reconciled. TS4 reports the contingency but disables boundary interpretation.

19.3 Framework-only boundaries

These are retained as diagnostics but should be absent under exact reconciled projection. Their presence requires inspection of source identity and projection semantics.

19.4 Atomic-only does not mean spectator-only

ATOMIC_ONLY means that the full atomic state changed while the projected framework class did not. The change may involve spectators, terminal atoms, or framework-local atomic details that do not alter the projected decorated graph. TS4 does not infer which mechanism applies.

19.5 Atomic transition does not imply site hopping

A Na–O contact change may occur within one adsorption basin. Site hopping requires later persistent ring-site or cage assignment.

19.6 Ensemble contingency is not a transition model

The contingency matrix is valid for ensembles, but no boundary classification, dwell time, return time, or rate is defined.

19.7 Per-frame identity modes

Per-frame atomic or framework identity deliberately avoids reconciliation. TS4 preserves static assignments but does not interpret adjacent public IDs as physical transitions.

19.8 Compression ratio

N_A/N_F is a catalog-count ratio. It is not a thermodynamic entropy, kinetic coarse-graining score, or proof that one model is physically superior.

20 Validation requirements

The TS4 implementation must test:

- exact frame-index alignment;
- exact frame-ID alignment;
- exact connectivity-state assignment alignment;
- topology representative source-digest alignment;
- two atomic states mapping to one framework class;
- several atomic states mapping to several framework classes;
- exact contingency counts and probabilities;
- deterministic dominant-state tie handling;
- stable, atomic-only, framework-only, and coupled boundary classification;
- ensemble boundary-statistics omission;
- per-frame identity-mode omission;
- option-controlled omission;
- read-only arrays;
- serialization round trip;
- digest tampering rejection;
- incorrect input types;
- Na-LTA integration behavior.

21 Na-LTA acceptance case

For the serialized 2,000-frame, 300 K Na-LTA catalogs, TS4 must report:

```
n_atomic_states = 72
n_framework_classes = 1
atomic_to_framework_compression_ratio = 72
atomic_states_per_framework_class = [72]
```

and, for the trajectory boundaries:

```
stable = 1928
atomic_only = 71
framework_only = 0
coupled = 0
```

The resulting regime must be

```
atomic_variable_framework_uniform
```

with the neutral interpretation:

```
atomic connectivity varies while framework topology remains uniform
```

22 Deferred features

The following remain outside TS4:

- approximate cross-catalog alignment;
- atom-identity mapping between independently ordered files;
- sparse contingency storage;
- mutual information and conditional entropy;
- transition probabilities and kinetic rates;
- Markov-state or hidden-state models;
- causality or mechanism inference;
- ring-level transition consequences;
- site-hop and cage-transition classification;
- plotting and table export, which belong to TS5.

23 Context-restoration checklist

Before revising TS4, confirm:

- Are both inputs authoritative catalogs over exactly the same selected frames?
- Do frame indices, frame IDs, and semantics agree exactly?
- Do framework connectivity-state assignments equal atomic state assignments?
- Are topology representatives derived from aligned atomic states?
- Is the framework catalog reconciled or per-frame?
- Is temporal boundary interpretation scientifically valid?
- Are atomic and framework ID namespaces kept separate?
- Is the contingency exact rather than approximately clustered?
- Are atomic-only events described neutrally?
- Is ensemble order being kept non-temporal?
- Are branch statistics reused rather than recomputed?
- Are schema versions and digests preserved?

24 Final contract

TS4 is the exact comparison layer between completed atomic and framework catalogs. It reports how atomic states project into framework classes and, where reconciled trajectory semantics exist, whether each atomic transition preserves or changes the framework.

It does not redefine either graph.